El Servicio Central y la Seguridad

Distribuidora de perecederos

Proyecto individual - Francisco Javier Sanchez Lopez / Javityan2 - matrícula 00000164512

Repositorio: https://github.com/Javityan2/Distribuidora-Perecederos
Rama de entrega: feat-servicio

Objetivo

Convertir el dominio de inventario de carnes en un servicio HTTP con persistencia MySQL, validación de entrada, autenticación y autorización por roles, conservando las reglas de negocio del avance anterior.

Arquitectura

Cliente HTTP > controllers y DTO > servicio de dominio > contrato InventarioRepository > repositorio MySQL > base de datos. Los guards verifican identidad y rol antes de ejecutar operaciones protegidas.

ProductosModule conecta los contratos de inventario y catálogo con la implementación MySQL mediante tokens. Los controllers delegan a servicios y no acceden a TypeORM. Los escenarios y servicios también se verifican con repositorios en memoria.

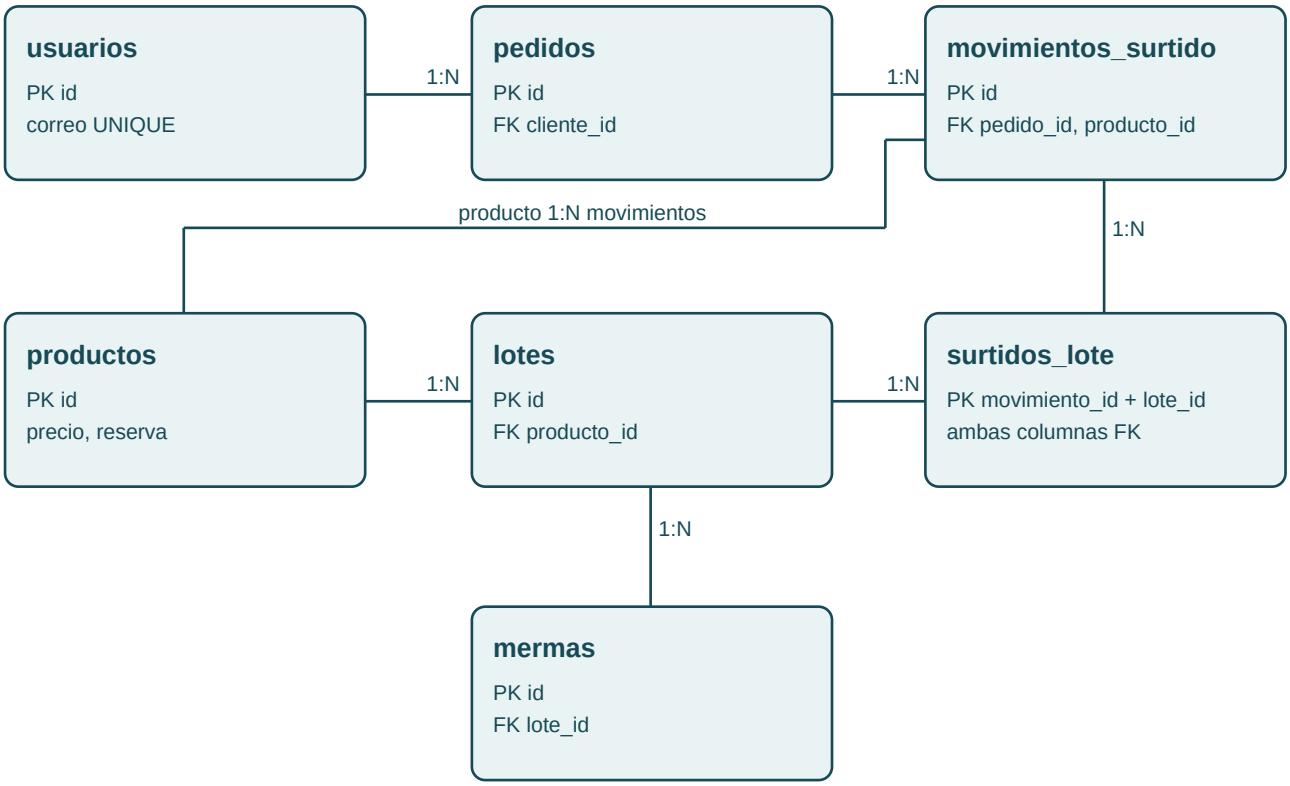
Tecnologías

NestJS 11, TypeORM, mysql2, MySQL 8, Argon2id, JWT HS256 y Swagger/OpenAPI. Las fechas se intercambian con zona horaria y se almacenan con conexión UTC.

Alcance

Catálogo, ingreso de lotes, creación y consulta de pedidos, surtido FEFO y mermas. Los estados en ruta y entregado existen en el modelo, pero no se agregaron casos de reparto. El proveedor se almacena como texto; el restaurante es un usuario con rol cliente.

1. Modelo de datos



Las conexiones 1:N indican que un registro de origen puede relacionarse con varios registros de destino. La pareja pedido-producto es única en movimientos_surtido.

Tabla	Atributos
usuarios	id INT PK; nombre VARCHAR(150); correo VARCHAR(254) UNIQUE; rol ENUM administrador, almacenista, cliente; activo BOOLEAN; password_hash VARCHAR(255).
productos	id INT PK; nombre VARCHAR(120); presentacion VARCHAR(100); unidad_medida ENUM kg, pieza, caja; precio_base DECIMAL(12,2); stock_minimo DECIMAL(12,3).
lotes	id INT PK; producto_id INT FK; proveedor VARCHAR(150); cantidad_disponible DECIMAL(12,3); fecha_ingreso y fecha_caducidad DATETIME(3).

2. Modelo, integridad e historial

Tabla	Atributos
pedidos	id INT PK; cliente_id INT FK a usuarios; fecha DATETIME(3); estado ENUM recibido, surtido, en ruta, entregado.
movimientos_surtido	id INT PK; pedido_id FK; producto_id FK; UNIQUE(pedido_id, producto_id).
surtidos_lote	movimiento_id FK y lote_id FK forman la PK compuesta; cantidad DECIMAL(12,3).
mermas	id INT PK; lote_id FK; cantidad DECIMAL(12,3); motivo ENUM caducidad, daño, faltante; fecha DATETIME(3).

Integridad

Los identificadores son INT UNSIGNED. Las claves foráneas evitan referencias inexistentes. CHECK impide saldos negativos, cantidades de merma/surtido no positivas y fechas de lote invertidas. Los precios y el stock mínimo no pueden ser negativos.

Historial reproducible

1. CrearInventario1792703804844 crea las siete tablas, relaciones y restricciones.
2. AgregarPasswordHash1792709127226 agrega la credencial transformada a usuarios.
TypeORM registra cada ejecución en migrations. synchronize está desactivado; las tablas no se alteran automáticamente al iniciar.

Transacciones

El repositorio usa aislamiento SERIALIZABLE. AsyncLocalStorage vincula los métodos al manager de la transacción actual. Ante cualquier error, se revierten saldos, movimientos y estado. Los conflictos de concurrencia se reportan como 409; se consulta el estado antes de reintentar.

Datos de arranque

npm run seed crea administrador, almacenista, restaurante, producto, dos lotes vigentes, uno caducado y un pedido. No sobrescribe credenciales ni saldos existentes. Las contraseñas son variables SEED_* del archivo local .env y no se incluyen en el repositorio.

3. Contrato HTTP y permisos

Método y recurso	Permiso
GET /	Público - identificación del servicio
GET /productos y /productos/:id	Público
POST /productos	Administrador
POST /auth/registro y /auth/login	Público
GET /auth/perfil	Usuario autenticado y activo
GET /usuarios	Administrador
GET /productos/:id/lotes; POST /lotes	Administrador / almacenista
GET /pedidos; POST /pedidos; GET /pedidos/:id	Personal: todos. Cliente: solo propios
POST /pedidos/:id/surtidos	Administrador / almacenista
GET /mermas; POST /mermas	Administrador / almacenista

Entradas principales

Registro: nombre, correo y password. Login: correo y password. Lote: productoId, proveedor, cantidadDisponible, fechaIngreso y fechaCaducidad. Pedido: clienteId. Surtido: lineas[{productoId,cantidad}]. Merma: loteId, cantidad y motivo. Los ejemplos y esquemas están disponibles en /docs.

Formato y estados

Éxito: {data: resultado, meta:{ruta,duracionMs,timestamp,requestId}}. Error: {data:null, error:{codigo,mensaje}, meta:{statusCode,path,timestamp,requestId}}. 200 consulta/login; 201 creación; 400 datos mal formados; 401 credencial ausente, inválida o vencida; 403 rol o propiedad incorrectos; 404 recurso inexistente; 409 regla de negocio o conflicto de datos.

Las cantidades numéricas inválidas para el negocio generan 409; los tipos incorrectos generan 400. Los errores SQL y secretos no se exponen. Un middleware genera X-Request-Id y registra método, ruta, estado y duración, incluidos los rechazos. El interceptor agrega el sobre uniforme y el filtro traduce ErrorDeDominio a HTTP. CORS permite únicamente los orígenes declarados en configuración.

4. Seguridad y evidencia

Autenticación y autorización

El registro público asigna cliente y activo. Las contraseñas se transforman con Argon2id y sal aleatoria; el hash se excluye de consultas habituales y de las respuestas. Login verifica credenciales y emite un JWT HS256 firmado, con 15 minutos de vigencia. El guard global verifica firma y vencimiento, consulta al usuario activo y toma su rol actual de MySQL. Las rutas públicas se marcan con @Publico().

El administrador crea productos y consulta usuarios. Administrador y almacenista ingresan lotes, surten pedidos y registran mermas. El cliente consulta catálogo y gestiona sus propios pedidos. No se permite escoger rol al registrarse ni acceder al pedido de otro cliente.

Pruebas reproducibles

Las pruebas unitarias verifican reglas de dominio, precisión de cantidades, identificadores y servicios con repositorios intercambiables. npm run test:mysql crea una base temporal aislada, aplica las dos migraciones y prueba el servicio HTTP real; finalmente elimina solo esa base. El reporte sin secretos se guarda en output/pruebas-avance2.json.

La integración verifica: registro válido/duplicado/inválido; login; token alterado y vencido; usuario inactivo; permisos 401/403/200; surtido FEFO; exclusión de caducados; doble surtido; insuficiencia y stock mínimo; rollback de pedidos con varios productos; mermas; concurrencia; contrato de usuarios; Swagger y CORS.

Instalación en otra computadora

README.md documenta la instalación de feat-servicio, las dependencias, la configuración local y los comandos migration:run, seed y start:dev. CORS_ORIGINS contiene los orígenes exactos de los clientes. Swagger está en http://localhost:3000/docs.

Secretos y operación

Las credenciales permanecen en .env, excluido del control de versiones. El ejemplo usa valores de relleno. El seed conserva datos previos; los lotes de demostración tienen caducidad relativa al primer seed. En una demostración posterior se registran lotes nuevos. Las pruebas de concurrencia pueden producir un 409 esperado en una de las solicitudes.